

Capitolul 7

Ecuatii diferențiale ordinare

7.1 Probleme cu valori inițiale

Metoda lui Euler este implementată în codul MATLAB de mai jos, care a fost scris într-o formă oarecum modulară pentru a scoate în evidență cele trei componente individuale. Programul care aproximează soluția apelează un subprogram pentru a executa fiecare pas Euler, care în schimb apelează o funcție care conține partea dreaptă f a ecuației diferențiale. În această formă, va fi ușor mai târziu să schimbăm partea dreaptă cu cea pentru o altă ecuație diferențială și metoda lui Euler cu o altă metodă mai sofisticată. Codul este dat mai jos.

Algoritm (Metoda lui Euler).

$$\begin{aligned}w_0 &= y_0 \\w_{i+1} &= w_i + hf(t_i, w_i).\end{aligned}$$

```
function [t,y]=euler1(inter,y0,n)
t(1)=inter(1);
y(1)=y0;
h=(inter(2)-inter(1))/n;
for i=1:n
    t(i+1)=t(i)+h;
    y(i+1)=eulerstep(t(i),y(i),h);
end
plot(t,y)
function y=eulerstep(t,y,h)
y=y+h*ydot(t,y);
function z=ydot(t,y)
z=t*y+t^3;
```

Programul poate fi apelat pentru a rezolva problema cu valoare inițială

$$\begin{cases} y' = ty + t^3 \\ y(0) = 1 \\ t \in [0, 1], \end{cases} \quad (7.1)$$

astfel:

```
>> [t,y]=euler1([0 1],1,10);
```

7.2 Sisteme de ecuații diferențiale ordinare

Codul MATLAB pentru rezolvarea sistemelor de ecuații diferențiale cu metoda lui Euler este foarte asemănător cu cel de mai sus, cu anumite ajustări pentru a trata pe y ca pe un vector:

```
function [t,y]=euler2(inter,y0,n)
t(1)=inter(1);
y(1,:)=y0;
h=(inter(2)-inter(1))/n;
for i=1:n
    t(i+1)=t(i)+h;
    y(i+1,:)=eulerstep(t(i),y(i,:),h);
end
plot(t,y(:,1),t,y(:,2));
function y=eulerstep(t,y,h)
y=y+h*ydot(t,y);
function z=ydot(t,y)
z(1)=y(2)^2-2*y(1);
z(2)=y(1)-y(2)-t*y(2)^2;
```

Pentru a rezolva sistemul

$$\begin{aligned}y_1' &= y_2^2 - 2y_1 \\ y_2' &= y_1 - y_2 - ty_2^2 \\ y_1(0) &= 0 \\ y_2(0) &= 1,\end{aligned}$$

folosim apelul

```
>> [t,y]=euler2([0 1],[0 1],10);
```

7.3 Metode cu pas variabil

Comanda MATLAB `ode45` folosește perechea încorporată Dormand–Prince împreună cu un protocol de control al mărimii pasului, după cum s-a descris în curs. Utilizatorul poate specifica toleranța relativă T dorită. Partea dreaptă a ecuației diferențiale trebuie specificată ca o funcție MATLAB. De exemplu, comenzile

```
>> opts=odeset('RelTol',1e-4,'Refine',1,'MaxStep',1);
>> [t,y]=ode45(@(t,y) t*y+t^3,[0 1],1,opts);
```

vor rezolva problema cu valoare inițială (7.1) și toleranță relativă a erorii $T = 0.0001$. Dacă parametrul `RelTol` nu este specificat, valoarea implicită 0.001 este folosită. Observăm că funcția f dată comenzii `ode45` trebuie să fie o funcție de două variabile, în acest caz t și y , chiar dacă una dintre ele nu apare în definiția funcției.

Pentru a vedea comanda `ode45` făcând selecția mărimii pasului, a trebuit să modificăm anumite setări prestabilite, folosind comanda `odeset`. Parametrul `Refine` mărește în mod normal numărul de valori ale soluției peste cele care sunt calculate de către metodă, pentru a face un grafic mai frumos, dacă și atunci când ieșirea este folosită cu acest scop. Valoarea prestabilită este 4, care face ca la ieșire să fie oferite de patru ori mai multe puncte decât sunt necesare. Parametrul `MaxStep` pune o limită superioară mărimii pasului h , având valoarea

implicită egală cu o zecime din lungimea intervalului. Folosirea valorilor prestabilite pentru ambii parametri ar însemna o mărime a pasului de $h = 0.1$, și după rafinarea cu un factor de 4, soluția ar fi prezentată cu o mărime a pasului de 0.025. De fapt, rulând comanda fără o variabilă de ieșire specificată, ca în codul

```
>> opts=odeset('RelTol',1e-6);
>> ode45(@(t,y) t*y+t^3,[0 1],1,opts);
```

va face ca MATLAB să reprezinte grafic automat soluția pe o grilă cu mărimea pasului constant egală cu 0.025.

O metodă alternativă de a defini funcția f care reprezintă partea dreaptă a unei ecuații diferențiale este de a crea un fișier de funcție, de exemplu `f.m`, și de a folosi caracterul `@` pentru a desemna function handle-ul ei:

```
function y=f(t,y)
y=t*y+t^3;
```

Comanda

```
>> [t,y]=ode45(@f,[0 1],1,opts);
```

face ca `ode45` să funcționeze ca mai înainte. Această alternativă va fi convenabilă când numărul de variabile independente din ecuația diferențială este mai mare.

7.4 Metode multipas

Următorul cod MATLAB folosește metoda trapezului pentru a da o valoare de start w_1 pentru cele trei metode Adams–Bashforth cu doi pași discutate în curs.

Algorithm (Metoda trapezului explicită).

$$\begin{aligned} w_0 &= y_0 \\ w_{i+1} &= w_i + \frac{h}{2}(f(t_i, w_i) + f(t_i + h, w_i + hf(t_i, w_i))). \end{aligned}$$

Algorithm (Metoda Adams–Bashforth cu doi pași).

$$w_{i+1} = w_i + h \left[\frac{3}{2}f(t_i, w_i) - \frac{1}{2}f(t_{i-1}, w_{i-1}) \right].$$

```
function [t,y]=exmultistep(inter,y0,n,s)
h=(inter(2)-inter(1))/n;
y(1,:)=y0;
t(1)=inter(1);
for i=1:s-1
    t(i+1)=t(i)+h;
    y(i+1,:)=trapstep(t(i),y(i,:),h);
    f(i,:)=ydot(t(i),y(i,:));
end
for i=s:n
    t(i+1)=t(i)+h;
    f(i,:)=ydot(t(i),y(i,:));
```

```

    y(i+1,:)=ab2step(t(i),i,y,f,h);
end
plot(t,y)
function y=trapstep(t,x,h)
z1=ydot(t,x);
g=x+h*z1;
z2=ydot(t+h,g);
y=x+h*(z1+z2)/2;
function z=ab2step(t,i,y,f,h)
z=y(i,:)+h*(3*f(i,+)/2-f(i-1,+)/2);
function z=unstable2step(t,i,y,f,h)
z=-y(i,+)/2+2*y(i-1,+)/2+h*(5*f(i,+)/2+f(i-1,+)/2);
function z=weaklystable2step(t,i,y,f,h)
z=y(i-1,+)/2+h*2*f(i,+)/2;
function z=ydot(t,y)
z=t*y+t^3;

```

Pentru a rezolva ecuația (7.1), codul de mai sus poate fi apelat astfel:

```
>> [t,y]=exmultistep([0,1],1,20,2);
```

O metodă simplă predictor–corector folosește metoda Adams–Bashforth explicită cu doi pași pe post de predictor cu metoda Adams–Moulton implicită cu un pas pe post de corector. Ambele sunt metode de ordinul doi. Codul MATLAB arată asemănător cu codul Adams–Bashforth folosit mai devreme, dar cu un pas corector adăugat:

Algoritm (Metoda trapezului implicită (de ordinul doi)).

$$w_{i+1} = w_i + \frac{h}{2}[f_{i+1} + f_i].$$

```

function [t,y]=predcorr(inter,y0,n,s)
h=(inter(2)-inter(1))/n;
y(1,:)=y0;
t(1)=inter(1);
for i=1:s-1
    t(i+1)=t(i)+h;
    y(i+1,:)=trapstep(t(i),y(i,:),h);
    f(i,:)=ydot(t(i),y(i,:));
end
for i=s:n
    t(i+1)=t(i)+h;
    f(i,:)=ydot(t(i),y(i,:));
    y(i+1,:)=ab2step(t(i),i,y,f,h);
    f(i+1,:)=ydot(t(i+1),y(i+1,:));
    y(i+1,:)=am1step(t(i),i,y,f,h);
end
plot(t,y)
function y=trapstep(t,x,h)
z1=ydot(t,x);
g=x+h*z1;
z2=ydot(t+h,g);

```

```

y=x+h*(z1+z2)/2;
function z=ab2step(t,i,y,f,h)
z=y(i,:)+h*(3*f(i,:)-f(i-1,:))/2;
function z=am1step(t,i,y,f,h)
z=y(i,:)+h*(f(i+1,:)+f(i,:))/2;
function z=ydot(t,y)
z=t*y+t^3;

```

Aceeași problemă (7.1) poate fi acum rezolvată astfel:

```
>> [t,y]=predcorr([0 1],1,20,2);
```

Probleme

1. Aplicați metoda lui Euler cu mărimea pasului $h = 0.1$ problemelor cu valoare inițială de mai jos. Afișați o matrice cu valorile t , aproximările date de metoda lui Euler, și erorile (diferența față de soluția exactă) la fiecare pas.

$$\begin{cases} y' = 1/y^2 \\ y(0) = 1 \\ t \in [0, 1], \end{cases}$$

$$\begin{cases} y' = 2(t+1)y \\ y(0) = 1 \\ t \in [0, 1], \end{cases}$$

$$\begin{cases} y' = t^3/y^2 \\ y(0) = 1 \\ t \in [0, 1]. \end{cases}$$

```
1.0000000000000000    1.177010584513025    -0.028060547574590
```

2. Reprezentați grafic aproximările date de metoda lui Euler pentru ecuațiile de la Problema 1, cu mărimea pasului $h = 0.1, 0.05$, și 0.025 , împreună cu soluția exactă.
3. Aplicați metoda trapezului explicită cu pasul $h = 0.1$ ecuațiilor de la Problema 1. Afișați o matrice cu valorile t , aproximările, și erorile globale de trunchiere la fiecare pas.

Algoritm (Metoda trapezului explicită).

$$\begin{aligned} w_0 &= y_0 \\ w_{i+1} &= w_i + \frac{h}{2}(f(t_i, w_i) + f(t_i + h, w_i + hf(t_i, w_i))). \end{aligned}$$

```
1.0000000000000000    1.206903979593307    0.001832847505692
```

4. Reprezentați grafic aproximările date de metoda trapezului explicită pentru ecuațiile de la Problema 1, cu mărimea pasului $h = 0.1, 0.05$, și 0.025 , împreună cu soluția exactă.

5. Aplicați metoda lui Euler cu mărimile pașilor $h = 0.1$ și $h = 0.01$ problemei cu valoare inițială de mai jos. Reprezentați grafic soluțiile aproximative pe $[0, 1]$.

$$\begin{aligned}y_1' &= y_1 + 3y_2 \\y_2' &= 2y_1 + 2y_2 \\y_1(0) &= 0 \\y_2(0) &= 1.\end{aligned}$$

1.0000000000000000 17.146072234500004 17.494750674600002

6. Aplicați metoda mijlocului cu mărimea pasului $h = 0.1$ pe $[0, 1]$ problemelor cu valoare inițială de la Problema 1. Afișați o matrice cu valorile t , aproximările, și erorile globale de trunchiere la fiecare pas.

Algoritm (Metoda mijlocului).

$$\begin{aligned}w_0 &= y_0 \\w_{i+1} &= w_i + hf\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}f(t_i, w_i)\right).\end{aligned}$$

1.0000000000000000 1.205144702731282 0.000073570643667

7. Aplicați metoda Runge–Kutta de ordinul patru cu mărimea pasului $h = 0.1$ pe $[0, 1]$ problemelor cu valoare inițială de la Problema 1. Afișați o matrice cu valorile t , aproximările, și erorile globale de trunchiere la fiecare pas.

Algoritm (Metoda Runge–Kutta de ordinul patru (RK4)).

$$w_{i+1} = w_i + \frac{h}{6}(s_1 + 2s_2 + 2s_3 + s_4)$$

unde

$$\begin{aligned}s_1 &= f(t_i, w_i) \\s_2 &= f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}s_1\right) \\s_3 &= f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}s_2\right) \\s_4 &= f(t_i + h, w_i + hs_3).\end{aligned}$$

1.0000000000000000 1.205072590231863 0.000001458144248

8. Aplicați comanda MATLAB `ode45` pentru problemele cu valoare inițială de la Problema 1.
9. Adaptați programul `exmultistep.m` pentru a aplica metoda Adams–Bashforth cu doi pași problemelor cu valoare inițială din Problema 1. Folosiți mărimea pasului $h = 0.05$ pe intervalul $[0, 1]$. Afișați o matrice cu valorile t , aproximările, și erorile globale de trunchiere la fiecare pas.

1.0000000000000000 1.204093354614963 -0.0009777777472652

10. Repetați Problema 9 folosind metoda instabilă cu doi pași din curs.

1.0000000000000000 -0.684859354262552 -1.889930486350167

11. Repetați Problema 9, folosind metoda Adams–Bashforth cu trei pași. Folosiți metoda Runge–Kutta de ordinul patru pentru a calcula w_1 și w_2 .

Algoritm (Metoda Adams–Bashforth cu trei pași (de ordinul trei)).

$$w_{i+1} = w_i + \frac{h}{12}[23f_i - 16f_{i-1} + 5f_{i-2}].$$

1.0000000000000000 1.205112379721514 0.000041247633899

12. Modificați programul `predcorr` într-o metodă de tip predictor–corector de ordinul trei, care folosește metoda Adams–Bashforth cu trei pași și metoda Adams–Moulton cu doi pași, având mărimea pasului 0.05. Afișați o matrice cu valorile t , aproximările, și erorile globale de trunchiere la fiecare pas.

Algoritm (Metoda Adams–Moulton cu doi pași (de ordinul trei)).

$$w_{i+1} = w_i + \frac{h}{12}[5f_{i+1} + 8f_i - f_{i-1}].$$

1.0000000000000000 1.205066966289011 -0.000004165798604

13. Modificați programul `predcorr` într-o metodă de tip predictor–corector de ordinul trei, care folosește metoda Adams–Bashforth cu patru pași și metoda Adams–Moulton cu trei pași, având mărimea pasului 0.05. Afișați o matrice cu valorile t , aproximările, și erorile globale de trunchiere la fiecare pas.

Algoritm (Metoda Adams–Bashforth cu patru pași (de ordinul patru)).

$$w_{i+1} = w_i + \frac{h}{24}[55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}].$$

Algoritm (Metoda Adams–Moulton cu trei pași (de ordinul patru)).

$$w_{i+1} = w_i + \frac{h}{24}[9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2}].$$

1.0000000000000000 1.205079126992707 0.000007994905092